

Reinforcement Learning Agents in Pac-Man

Kim Svedberg*, Zebastian Thorsén†, Nadja Säfströmer‡, Antonio Vaca Landeta§

School of Innovation, Design and, Engineering
Mälardalens University, Västerås, Sweden

Email: *Ksg22001, †Ztn22002, ‡Nsr20002, §Ava17002@student.mdu.se

Abstract— Reinforcement learning has shown strong potential for learning effective policies in complex, stochastic game environments. This paper implements and compares three reinforcement learning approaches in the game of Pac-Man: Approximate Q-learning (AQL), a standard Deep Q-learning agent, and a CNN-based Deep Q-Network (DQN). Each agent was trained from scratch and evaluated over 50 episodes against a random baseline. All agents outperformed the random baseline, with the DQN achieving the best overall results: an average of 59.5% of pellets collected and a mean survival time of 28.3s per episode. The AQL and Deep Q-learning agents achieved 29.1% and 42.6% respectively. None of the agents were able to clear the full board, suggesting that further training, improved exploration strategies, and richer feature representations are still required.

Index Terms—Approximate Q-learning, DQN, Deep Q-learning, Reinforcement Learning, RL, Pac-Man, Q-learning

I. INTRODUCTION

Pac-Man is a classic arcade game which a player navigates a maze, collecting pellets while avoiding pursuing ghosts. Despite its simple premise, Pac-Man presents a complex control problem for a autonomous agents: the optimal strategy requires both short-term evasion and long-term planning, and the ghost behavior introduces stochasticity that demand adaptive responses [1]. For these reasons, Pac-Man projects can be found where reinforcement learning (RL) is applied in the game with the purpose of teaching foundational AI concepts such as RL, informed state-space search, and probabilistic inference. These concepts are important for real-world fields, including computer vision, robotics, and natural language processing [2].

Reinforcement learning enables an agent to learn optimal behavior through interaction with an environment by maximizing cumulative reward [3]. Among RL approaches, value based methods such as Q-learning have been widely used due to its simplicity and application in discrete state spaces. In Q-learning, the agent maintains a table of action-value estimates $Q(s, a)$ and updates them using the Bellman equation after each observed transition. However, classical tabular Q-learning becomes impractical when applied to environments with large or continuous state spaces, such as video games, where the state space can grow exponentially.

One approach that tries to address this that has attracted attention in recent years is Approximate Q-learning (AQL).

This method can solve nonlinear optimal control problems when the model of the plant is not known. In AQL, the algorithm may provide a near-optimal solution due to function approximation errors. However, when the state and action spaces are continuous, updating the Q-function for all states and actions becomes impractical. For that reason, instead of updating all values of the Q-function, the values are estimated by a function estimator, AQL [4]. In cases where state and action spaces are too large or complex for manual feature engineering, Deep Q-learning integrates Q-learning with deep neural networks to approximate the action-value function directly from high-dimensional input.

A. Related works

Studies related to Approximate Q-learning, apply a similar approach where the Q-value ($Q(s, a)$) is estimated from a linear combination of features extracted from the state-action representation ($f_i(s, a)$), including a bias component. The objective is for the Pac-Man agent to learn the weights associated with the features extracted from the game states [5].

The configurations for the algorithm involve well-chosen hyperparameters to balance exploration and learning stability. This included starting with a higher exploration rate ϵ and gradually reducing it, while maintaining a stable discount factor γ [6]. In related AQL studies the algorithm was applied to smaller grids. The results indicate that AQL performs well in smaller environments, and can improve learning performance compared to SARSA [5]. Another important aspect highlighted is the importance of feature extraction, which allows agents to learn better in larger state spaces, where traditional tabular Q-learning methods tend to fail [6].

Mnih et al. [7] introduced the Deep Q-Network (DQN), a RL agent that combines Q-learning with a deep convolutional neural network (CNN) to learn control policies directly from raw pixel input. The main objective of the DQN is to approximate the optimal action value function $Q^*(s, a)$ using a deep CNN that takes stacked preprocessed game frames as input and outputs Q-values for each possible action. To address the well-known instability of training neural networks with Q-learning, two mechanisms were introduced: experience replay, which stores past transitions and samples them randomly during training to break temporal correlations, and a separate target network whose weights are held fixed for a number of steps to produce stable training targets. The agent was evaluated on 49 Atari 2600 games using identical architecture and hyperparameters across all games, achieving performance

comparable to a professional human tester on the majority of them.

The standard Q-learning algorithm is known to overestimate action values under certain conditions. Hasselt et al. [8], demonstrated that this problem carries over to the DQN algorithm, producing substantial overestimations in games from the Atari 2600 domain. Their proposed solution, Double Q-learning, decouples the selection of the best next action from its evaluation: the online network selects the greedy action in the next state, while a separately maintained target network evaluates it. This resulted in reduced overestimation bias and led to considerably better performances across several games.

The aim of this paper is to implement and compare three different reinforcement learning approaches (AQL, Deep Q-learning and DQN) for controlling an autonomous Pac-Man agent. This raises the question:

• **RQ1:** Which of the three agents is best by evaluating metrics such as pellets eaten and survival time in seconds to play the Pac-Man game?

II. METHOD

Three reinforcement learning algorithms were implemented and evaluated on the Pac-Man environment: Approximate Q-learning with linear function approximation, a standard deep Q-learning network using hand-crafted feature vectors, and a Deep Q-Network (DQN) with convolutional feature extraction from raw frames.

The base Pac-Man game was implemented in Python 3 using the Pygame library following a guide [9] running at 30 FPS, See Figure 1 for the maze layout. At each time step, the agent observes the current game state, selects one of four discrete actions (UP, DOWN, LEFT, RIGHT), and receives a reward or penalty. The agent's objective is to maximize the cumulative reward over an episode.

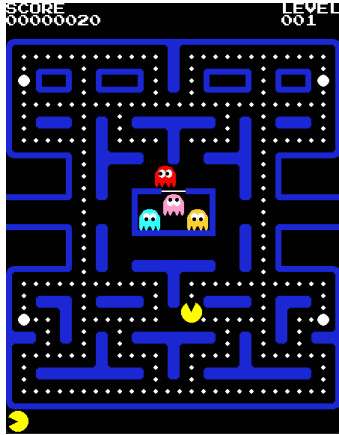


Figure 1. Classic Pac-Man maze used for all RL approaches.

A. Approximate Q-learning

For the implementation of Approximate Q-Learning (AQL), several features were hand-crafted. The selected features include the normalized distance to the nearest food, ghost

proximity indicators, for example, whether a ghost is one step away, and the distance to the closest active ghost. Together with other features such as nearby scared ghosts, fruit proximity, remaining pellets, and action-indicator features, these allow the Pac-Man agent to capture important data from the environment, motivating the agent to avoid dangerous ghosts and eat food in order to obtain rewards.

In AQL, the Q-value is approximated as a linear combination of features extracted from the state-action pair:

$$Q(s, a; w) = \sum_{i=1}^n f_i(s, a) w_i [5] \quad (1)$$

where $f_i(s, a)$ represent feature values and w_i the weights. The weights are updated using the following rule:

$$w_i \leftarrow w_i + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right) f_i(s, a) [5] \quad (2)$$

where $\gamma = 0.9$ represents the discount factor.

In addition, a bias term $f_0(s, a)$ with a constant value of 1 was included [5].

Furthermore, the algorithm was first trained for 2000 episodes, where each life represents one episode. During training, an $\epsilon - greedy = 0.3$ was used for exploration for both the initial training of 2000 episodes and the final training run of 5000 episodes. In the last training phase, $\epsilon - greedy = 0.5$ was used for greater exploration, and an $\epsilon - decay = 0.9$ was applied in order to reduce the exploration gradually to a minimum value of 0.1.

B. Standard Deep Q-learning

With the help of Tensor-Flow library in python a deep Q-learning algorithm was implemented. Deep Q-learning is a reinforcement algorithm that learns by receiving rewards. In this application of the deep Q-learning instead of using CNN for processing it uses feature vectors containing information of the positions of the Pac-Man and distances to objects such as ghosts and pellets.

The training process involves looking at the current state and selecting an action based on a epsilon-greedy policy, this means that with a probability of ϵ the Deep Q-learning algorithm will select a random action, and with probability $1 - \epsilon$ the action with the highest predicted Q-value is chosen [7].

For the standard Deep Q-learning method, an approach with Double Q-learning was chosen. The method separates the choice of the best action and evaluates how good it is. This results in the main network calculating the best possible action and the target network evaluates that action, this is to ensure that the best actions are taken [8].

The target value for the algorithm is calculated by using the following equation:

$$y = r + \gamma \times Q_{\text{target}} \left(s', \arg \max_a Q(s', a) \right) \quad (3)$$

where r is the reward, γ is the discount factor, a is the action, and s' is the next state.

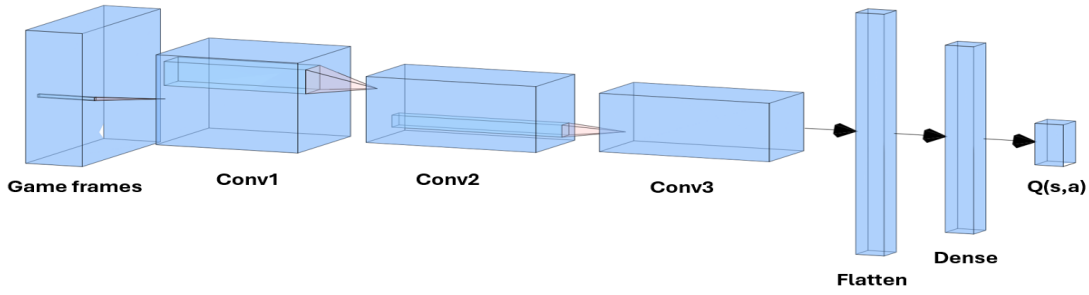


Figure 2. Architecture of the DQN. The network processes stacked grayscale frames through three convolutional layers, followed by a flatten layer and a fully connected layer, to output Q-values for four actions.

The architecture of the model is two dense layers with rectified linear unit activation function (ReLU) and a output layer. The following is an overview of the Standard Deep Q-learning architecture:

- **Layers:**
 - Layer 1: 256 neurons, ReLU activation.
 - Layer 2: 256 neurons, ReLU activation.
- **Output Layer:**
 - Output layer with 4 neurons

A replay buffer was used to store previous movements, which helps improve stability during training.

C. DQN

The Deep Q-Network (DQN) was implemented using TensorFlow to approximate the optimal action-value function for Pac-Man. The network takes preprocessed game frames as input and outputs Q-values for the four possible actions at each state: UP, DOWN, LEFT and RIGHT.

The DQN architecture consists of:

- **Convolutional layers:** Three sequential convolutional layers extract spatial features from the game state:
 - Layer 1: 32 filters, 8x8 kernel, stride 4, ReLU.
 - Layer 2: 64 filters, 4x4 kernel, stride 2, ReLU.
 - Layer 3: 64 filters, 3x3 kernel, stride 1, ReLU.
- **Flattening layer:** Converts the 3D feature maps to a 1D vector
- **Dense layers:**
 - Fully connected layer with 512 neurons and ReLU activation
 - Output layer with 4 neurons (linear activation) producing Q-values for each action

This architecture is inspired and based from [7] and [5], which has proven effective for learning policies from visual input, see Figure 2. The following techniques were used to stabilize training:

- 1) **Frame preprocessing:** Each raw game frame was converted to an 84x84 grayscale image. Four consecutive frames were then stacked along the channel dimension to form the state representation (84x84x4), allowing the network to infer motion and velocity.

- 2) **Experience replay:** A replay buffer storing the 50,000 most recent transitions (s, a, r, s') was used, where s is the current state, a is the action taken, r is the reward received, and s' is the next state. Transitions were stored after every 4 frames to reduce temporal correlation.
- 3) **Mini-batch sampling:** At each training step, a mini-batch of 64 transitions was randomly samples from the replay buffer to compute the loss.
- 4) **Target network:** A separate target network Q_{θ^-} with identical architecture to the online network Q_{θ} was used to generate stable target Q-values. The target network parameters were hard-copied from the online network every 5,000 training steps, reducing oscillations in the learning targets.
- 5) **Double Q-learning:** To minimize overestimation bias, Double Q-learning was implemented, which decouples the action selection from the evaluation [8], using the same equation (3) as the Deep Q-learning model.
- 6) **Epsilon-greedy exploration:** An ϵ -greedy policy balanced exploration and exploitation. ϵ decayed exponentially from initially 1.0 (fully random) to a minimum of 0.05, with a decay factor of 0.99999. This allowed the agent to explore thoroughly early in training while increasingly exploiting learned behaviors later on.

The DQN was trained using the Adam optimizer with Huber loss. The learning rate was initially set to 1×10^{-4} and gradually reduced to 1×10^{-5} in later training stages to fine-tune the learned policy. Gradient clipping with a norm of 10.0 was applied to prevent destabilizing large updates. Training began after an initial warm-up period of 5,000 random steps to populate the replay buffer.

D. Reward Structure

All three approaches shared the same reward scheme, with agent specific scaling. Positive rewards were assigned for collecting pellets, power pellets, and fruit, eating a scared ghost, and clearing the full maze. A time-step penalty was applied at every step to discourage stalling, and a large negative reward was issued upon death to strongly penalize ghost collisions.

E. Evaluation

After training, all three agents were evaluated over 50 episodes in inference mode, to measure the performance of the learned policy. A random agent, selecting actions at random, was also evaluated to serve as a baseline comparison.

Two metrics were used to assess agent performance: the number of pellets collected per episode and the game duration in seconds. Pellets collected serves as the primary measure of task completion, as clearing the maze of all pellets is the core objective of Pac-Man. It reflects how effectively the agent navigates the environment and avoids dying before collecting food. Game duration complements this by capturing survival ability: an agent that dies quickly will have fewer opportunities to collect pellets regardless of its policy quality, so duration helps distinguish between agents that perform well due to competent play versus other factors. Raw score was not selected as a metric because the reward functions were tuned differently across agents during training, making direct score comparisons potentially misleading.

III. RESULTS

A. Approximated Q-learning

The training result of the Approximated Q-learning after 5000 episodes can be seen in Figure 3 and 4. The learned weights show the behavior of the agent.

- Food Closeness: The weight for Food Closeness was strongly positive (+236), which can be interpreted as the agent having learned to prioritize moving toward nearby food.
- Ghost One Step: The weight is strongly negative (-256), which shows that the agent has learned to avoid the ghost that is right next to the agent, helping to reduce the number of deaths. Expected based on the high penalty value.
- Ghost In Front: The weight is less negative (-23), which contributes to avoiding dangerous situations.
- Scared Ghost: The weight is slightly negative(-13), which shows that the agent does not strongly prioritize scared ghosts, the agent focuses on playing safer and obtaining rewards from eating pellets.
- Food In Direction: The weight is positive (+17), the agent prefers directions where there are more pellets.

B. Standard Deep Q-learning

The Standard Deep Q-learning model was unsuccessful in completing a complete game (eat all pellets) without dying. However, it showed promising results in finding paths to try to avoid the ghosts.

Figure 5 shows how many pellets were eaten during training. The model does show some increase of the number of pellets eaten for each episode and the image shows that during training number of pellets eaten grows to a more stable number. Figure 6 shows a slight increase of the survival time in seconds for the agent during training.



Figure 3. Number of pellets collected per episode by the AQL. The orange line shows the 30-episode moving average. The best single episode collected approximately 200 pellets.



Figure 4. Episode duration in seconds by the AQL. The orange line shows the 30-episode moving average. It can be observed that the duration is higher compared to random policy, and it improves over episodes.

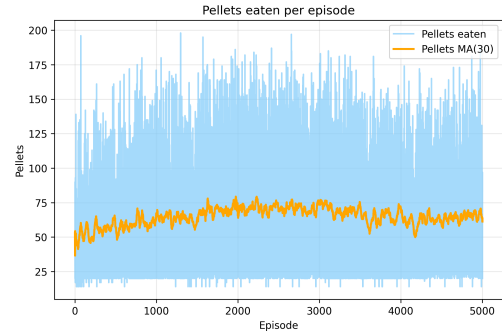


Figure 5. Number of pellets collected per episode by the Deep Q-learning agent. The orange line shows the 30-episode moving average.

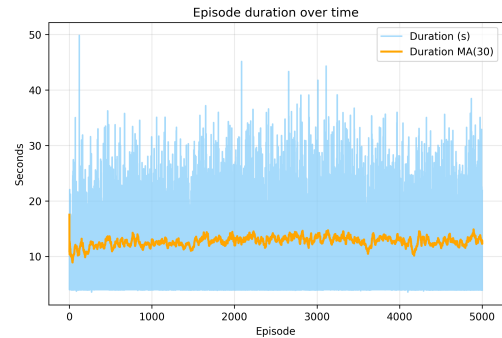


Figure 6. Episode duration in seconds by the Deep Q-learning agent. The orange line shows the 30-episode moving average.

C. Deep Q-Network

The DQN agent was trained for 6800 episodes/games but was unable to successfully complete a single game (i.e., clear all 244 pellets from the board).

Figure 7 shows the number of pellets eaten per episode over the course of training. The DQN agent demonstrated clear learning progress, improving from an initial average of approximately 20-30 pellets in early episodes to 125-140 pellets after 5000 episodes, which is about 52-58% of the available pellets on the board. The best single-episode performance achieved 196 pellets (81.7% of the board).

Figure 8 shows the episode length over the course of training. The DQN agent showed steady improvement in survival time, increasing from an initial average duration of 11 s per episode to over 22 s after 3000 episodes.

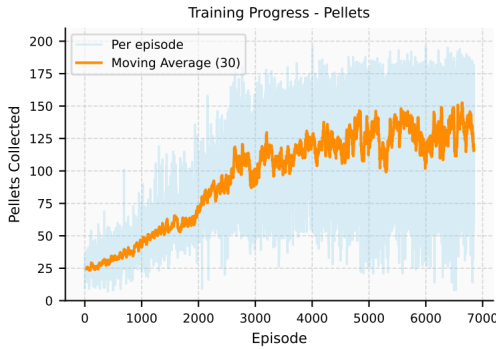


Figure 7. Number of pellets collected per episode by the DQN. The orange line shows the 30-episode moving average.

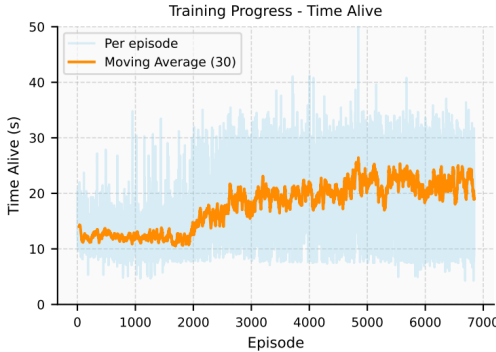


Figure 8. Episode duration in seconds by the DQN. The orange line shows the 30-episode moving average.

D. Evaluation of Trained Models

Figure 9 and 10 compare the 50 episode average evaluation performance of each trained agent against a random baseline in average pellets eaten and average game duration. All three agents outperformed the random baseline in pellets eaten. The DQN achieved the highest average of 145.2 pellets per episode, representing 59.5% of the 244 available pellets on the board. The standard deep Q-learning model averaged 103.9 pellets (42.6%), and AQL averaged 71 pellets (29.1%), more

than double the random baseline of 24.9 pellets (10.2%). A similar order of results can be seen in game duration. The DQN survived for an average of 28.3 s per episode, compared to 17.9 s for the Deep Q-learning, 20 s for AQL, and 14.6 s for the random agent.

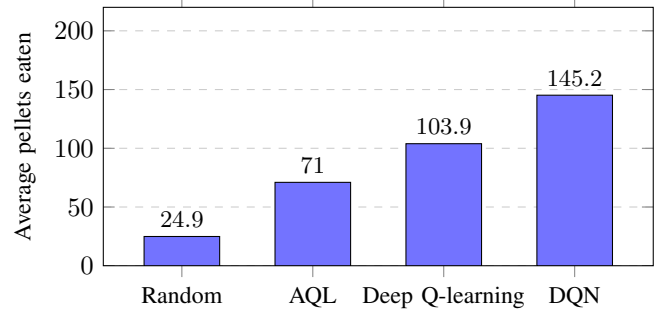


Figure 9. Average number of pellets eaten per episode over 50 evaluation episodes ($\epsilon = 0$) for each agent. Higher is better; the maze contains a total of 244 pellets.

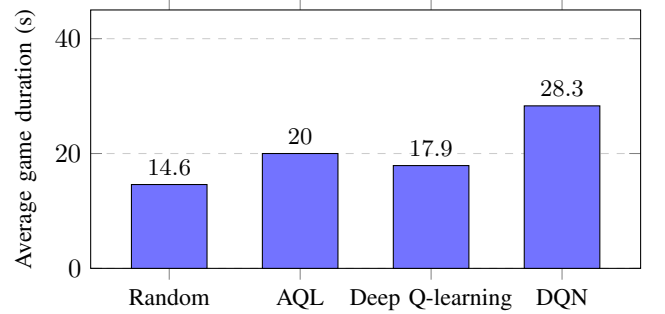


Figure 10. Average game duration in seconds per episode over 50 evaluation episodes ($\epsilon = 0$) for each agent. Higher is better; a longer survival time indicates a more effective avoidance and navigation strategy.

IV. DISCUSSION

None of the models tested were successful in clearing the entire Pac-Man board of pellets. This was most likely due to the limited training time (episodes/games) and limitations in the hardware. Reinforcement learning methods typically require a large number of training episodes to learn complex patterns and strategies, which is relevant in the Pac-Man game where there is high variability in pattern. If hardware with better parallelization and processing power is used, one may be able to shorten the time required for each episode and hence increase the total number of games it is trained for, which could increase the agents performance.

As seen in Figure 7, the training of the DQN was most effective up til 5000 episodes, where it started to level out. This suggests that the model might not have fully converged to an optimal state, but instead achieved an equilibrium locally at that point. This may be due to insufficient exploration with too quick decay of the epsilon-greedy strategy, causing the agent to stop exploring and instead exploiting policies which are not optimal. This problem aligns with previous research

on exploration vs exploitation, where insufficient exploration can cause worse performance by not finding the optimal policy [10]. Also visible was a high variance in performance due to the random initialization of neural network weights, leading to unstable learning of the DQN and Deep Q agent. The variability is a known behavior of reinforcement learning and has been demonstrated in multiple papers [3], [7].

With improved hardware and longer training time, one could more easily explore different parameterizations of learning rate, epsilon decay, and network layers to allow the model to explore more thoroughly and prevent early convergence while achieving improvements in performance.

Among the methods used as agents, the one that achieved the best result in terms of most pellets eaten was the Deep Q-Network with CNN. It achieved an average score of approximately 59.5 % of the board, compared to AQL and Deep Q-learning which both achieved lower performance. The result is in accordance with other studies showing that DQN and other deep architecture models generally outperform simpler agents when tasked with controlling a high dimensional game.

The AQL agent, relying on linear function approximation over a hand-crafted five-feature vector, achieved 29.1% of pellets, more than double the random baseline of 10.2%. Strong positive weight on food closeness, strong negative weight on adjacent ghosts, and a moderate positive weight on food-dense directions all show a reasonable survival and pellet collection strategy. However, the representational capacity of a linear model over five features is limited. Interactions between features, long-range spatial dependencies, and multi-step planning are difficult or impossible to capture without richer representations or non-linear function.

The standard deep Q-learning agent achieved a few episodes with a pellet count higher than that of DQN, but because it had a higher variance in performance, it did not outperform DQN in average performance. A reason to why DQN performed better may be that it uses both convolutional layers and frame preprocessing for visual input, allowing the agent to extract spatial information directly from the visuals, avoiding the need of manual feature extraction while reducing noise. For the AQL agent to perform as well, it would likely require improved manual feature extraction and the use of more advanced adaptive methods in training. Good feature extraction is fundamental to capture and represent the environment accurately, allowing the model to learn an optimal policy. Similarly, the Deep Q-learning without CNN is also insufficient in capturing spatial information of the game, leading to poorer average performance and policy than the DQN.

A. Future Work

Several directions could be taken to improve the results. First, training all agents for more episodes would allow for continued policy improvement and a more reliable convergence assessment. Second, a better approach to exploration could mitigate premature convergence. Adaptive epsilon decay, entropy regularization, or reward bonuses for states are worth investigating. Third, the reward scaling used across agents were designed

independently and are not directly comparable. Standardizing reward scaling across all agents would improve the validity of agent comparisons and could improve performance by shaping the reward signal more carefully. Fourth, the feature set used for the AQL and Deep Q-learning agents is a clear bottleneck. Testing and adding additional features, such as the distance to the nearest intersection or the number of remaining pellets in each quadrant, could perhaps improve their representational capacity. Finally, architectural extensions to the DQN are worth exploring. Combining the CNN with recurrent layers, like a Long short-term memory (LSTM), would allow the agent to maintain a memory of recent states, which could be especially useful for anticipating ghost moves and planning longer sequences of moves.

V. CONCLUSION

In conclusion, all agents demonstrated clear learning progress during training and outperformed the random baseline in both pellets collected and survival time. The DQN achieved the best overall performance, collecting an average of 59.5% of available pellets and surviving 28.3s per episode, due to its convolutional architecture and ability to extract features directly from visual input. The standard Deep Q-learning agent achieved 42.6% of pellets, while AQL achieved 29.1%, more than double the random baseline, demonstrating that even a linear agent with well-designed features can learn a meaningful policy.

None of the agents were able to clear the full board, showing the inherent difficulty of the task and the constraints of limited training time and hardware. Future improvements through further training, improved exploration strategies, and richer feature representations are needed. Overall, the results show that reinforcement learning, and the DQN architecture in particular, provides a strong foundation for learning effective game playing agents in complex, stochastic environments.

ACKNOWLEDGMENT

The authors would like to thank Ning Xiong for his help and support during the process of developing the models and writing this paper. The authors would also like to thank Jonathan Richards for providing a step by step guide on his website for how to implement the base Pac-Man game in Python using Pygame [9].

VI. CONTRIBUTION

Kim Svedberg: Coded and customized the base Pac-Man game and implemented the DQN.

Zebastian Thorsén: Implemented the Deep Q-learning.

Nadja Säfströmer: Implemented the Deep Q-learning.

Antonio Vaca Landeta: Implemented the Approximate Q-learning.

REFERENCES

- [1] P. Rohlfschagen, J. Liu, D. Perez-Liebana, and S. M. Lucas, "Pac-man conquers academia: Two decades of research using a classic arcade game," *IEEE Transactions on Games*, vol. 10, no. 3, pp. 233–256, 2018.

- [2] P. A. e. a. John DeNero, Dan Klein, "The pac-man projects," 2025, accessed: 03-09-2026. [Online]. Available: <https://inst.eecs.berkeley.edu/~cs188/fa25/projects/>
- [3] R. S. Sutton and A. G. Barto, "Reinforcement learning: An introduction, second edition," accessed: 12-03-2026. [Online]. Available: <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>
- [4] Y. Li, C. Yang, Z. Hou, Y. Feng, and C. Yin, "Data-driven approximate q-learning stabilization with optimality error bound analysis," *Automatica*, vol. 103, pp. 435–442, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0005109819300251>
- [5] J. A. Abeynaya Gnanasekaran, Jordi Feliu Faba, "Reinforcement learning in pacman," accessed: 01-03-2026. [Online]. Available: <https://cs229.stanford.edu/proj2017/final-reports/5241109.pdf>
- [6] S. Theanraj, "Comparative study of reinforcement learning using pacman game," accessed: 11-03-2026. [Online]. Available: https://cis.temple.edu/~pwang/5603-AI/Project/2025F/Theanraj/Pacman_reinforcement_learning.pdf
- [7] K. K. S. D. e. a. Mnih, V., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.
- [8] H. Van Hasselt and A. Guez, "Deep reinforcement learning with double q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, 09 2015. [Online]. Available: https://www.researchgate.net/publication/282182152_Deep_Reinforcement_Learning_with_Double_Q-Learning
- [9] J. Richards, "Pacmancode," <https://pacmancode.com/>, accessed: 01-03-2026.
- [10] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, no. 2-3, pp. 235–256, 2002.